

UNIVERSIDAD DE LAS FUERZAS ARMADAS



“ESPE”

DEPARTAMENTO DE CIENCIAS DE LA COMPUTACIÓN

INGENIERÍA EN ANÁLISIS Y DISEÑO DE SOFTWARE

CARRERA DE INGENIERÍA EN SOFTWARE



NOMBRE:

- Blacio Machuca Julio César
- Sigcha Palma Kenned Jhair
- Solano Sanchez Josue Vital
- Toapanta Jama Alexander Jhosue

NRC: 30724

FECHA: 28/04/2026

DOCENTE: Ing. Jenny Ruiz Robalino

Diagrama de clases de análisis

```
@startuml
left to right direction
skinparam packageStyle rectangle
skinparam actorStyle awesome

' =====
' ACTORES
' =====

actor "Administrador" as Admin
actor "Entrenador" as Ent
actor "Atleta" as Atl
actor "Sistema de Pagos" as SP <<System>>

' =====
' FRONTERAS (Boundary)
' =====

boundary "Pantalla\nAgenda Clases" as UI_ADM1
boundary "Pantalla\nMembresías" as UI_ADM2
boundary "Pantalla\nRendimiento" as UI_ENT1
boundary "Pantalla\nReservas" as UI_ATL1

' =====
' CONTROLES (Control)
' =====

control "Control\nClases" as Ctrl_ADM1
control "Control\nMembresías" as Ctrl_ADM2
control "Control\nRendimiento" as Ctrl_ENT1
control "Control\nReservas" as Ctrl_ATL1

' =====
' ENTIDADES (Entity)
' =====

entity "Clase\n- día\n- hora\n- duración\n- cupo máximo\n- entrenador" as E_Clase
entity "Membresía\n- nombre\n- precio\n- características\n- estado\n- fecha vencimiento"
as E_Membresia
entity "Rendimiento\n- tiempo\n- repeticiones\n- peso\n- puntuación\n- atleta" as
E_Rendimiento
entity "Reserva\n- clase\n- horario\n- entrenador\n- estado" as E_Reserva

' =====
' RELACIONES CUC-ADM-01
' =====

Admin --> UI_ADM1 : gestiona clases
UI_ADM1 --> Ctrl_ADM1 : envía datos de clase
Ctrl_ADM1 --> E_Clase : CRUD clase
```

' =====

' RELACIONES CUC-ADM-02

' =====

Admin --> UI_ADM2 : gestiona membresías

UI_ADM2 --> Ctrl_ADM2 : envía datos de membresía/pago

Ctrl_ADM2 --> E_Membresia : CRUD membresía

Ctrl_ADM2 --> SP : procesa pago

' =====

' RELACIONES CUC-ENT-01

' =====

Ent --> UI_ENT1 : registra rendimiento

UI_ENT1 --> Ctrl_ENT1 : envía resultados WOD

Ctrl_ENT1 --> E_Rendimiento : CRUD rendimiento

' =====

' RELACIONES CUC-ATL-01

' =====

Atl --> UI_ATL1 : reserva clases

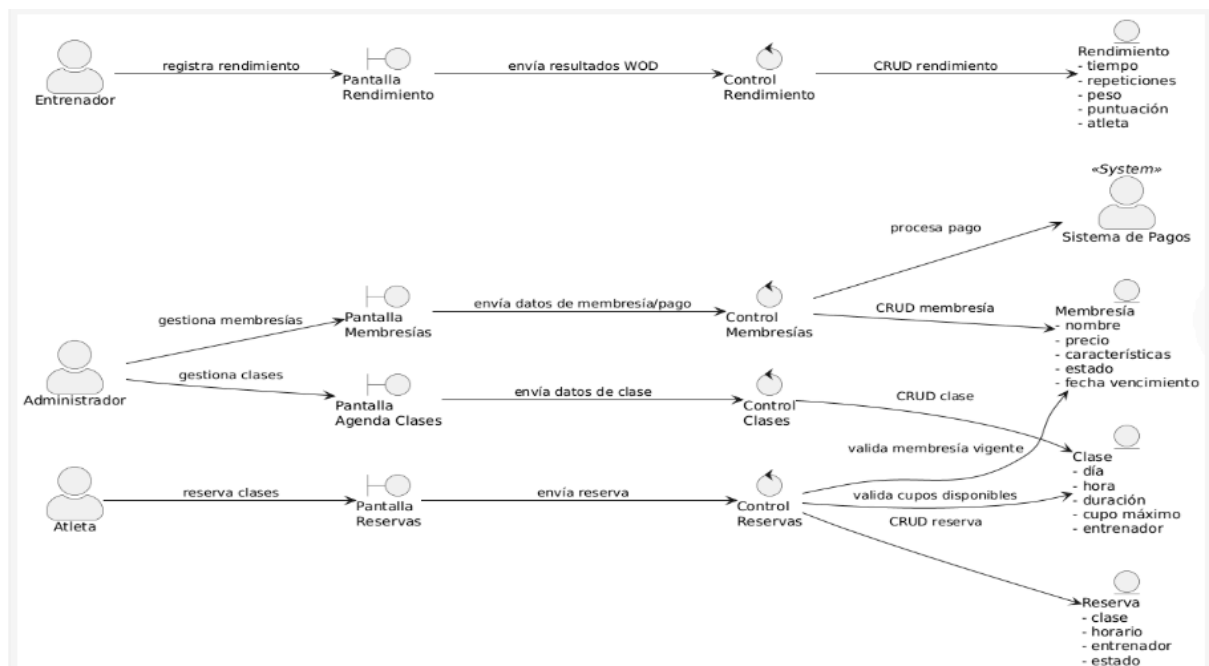
UI_ATL1 --> Ctrl_ATL1 : envía reserva

Ctrl_ATL1 --> E_Reserva : CRUD reserva

Ctrl_ATL1 --> E_Membresia : valida membresía vigente

Ctrl_ATL1 --> E_Clase : valida cupos disponibles

@enduml



Matriz de relación: casos de uso, clases y secuencias

Caso de uso	Boundary	Control	Entity / Repositorio	Diagrama de secuencia esperado
CUC-ADM-01 Administrar la agenda de clases	PantallaAgendaClases captura día, hora, duración, cupo máximo y entrenador desde el formulario.	ControlClases valida no solapamiento de horarios, verifica disponibilidad del entrenador y coordina el guardado.	Crea Clase y la guarda en RepositorioClase.	Secuencia: ingresar datos de clase → validar no solapamiento → verificar disponibilidad del entrenador → crear entidad Clase → guardar → confirmar.
CUC-ADM-01 Modificar clase existente	PantallaAgendaClases captura ID de clase y nuevos valores.	ControlClases busca existencia, valida nueva disponibilidad y coordina actualización.	Actualiza Clase existente en RepositorioClase.	Secuencia: seleccionar clase → cargar datos actuales → modificar datos → validar disponibilidad → actualizar entidad → confirmar.
CUC-ADM-01 Eliminar clase	PantallaAgendaClases solicita selección de clase a eliminar.	ControlClases verifica existencia y confirma eliminación.	Elimina el registro de Clase en RepositorioClase.	Secuencia: seleccionar clase → confirmar eliminación → eliminar entidad → liberar

				horario → notificar.
CUC-AD M-01 Mostrar agenda de clases	PantallaAgendaClases solicita mostrar lista y presenta tabla semanal.	ControlClases solicita todos los registros.	RepositorioClase devuelve lista de Clase con día, hora, entrenador y cupos.	Secuencia: solicitar agenda → recuperar lista de clases → mostrar tabla semanal.
CUC-AD M-02 Crear tipo de membresía	PantallaMembresias captura nombre, precio y características desde el formulario.	ControlMembresias valida datos obligatorios y coordina el guardado.	Crea Membresia y la guarda en RepositorioMembresia.	Secuencia: ingresar datos de membresía → validar campos obligatorios → crear entidad Membresía → guardar → confirmar.
CUC-AD M-02 Modificar tipo de membresía	PantallaMembresias captura ID de membresía y nuevos valores.	ControlMembresias busca existencia y coordina actualización.	Actualiza Membresia existente en RepositorioMembresia.	Secuencia: seleccionar membresía → cargar datos actuales → modificar datos → validar → actualizar entidad → confirmar.

CUC-ADM-02 Eliminar tipo de membresía	PantallaMembresias solicita selección de membresía a eliminar.	ControlMembresias verifica que no esté asignada a atletas activos y confirma eliminación.	Elimina el registro de Membresia en RepositorioMembresia.	Secuencia: seleccionar membresía → validar sin atletas activos → confirmar eliminación → eliminar entidad → notificar.
CUC-ADM-02 Registrar pago de atleta	PantallaMembresias captura atleta, monto y método de pago.	ControlMembresias procesa transacción con Sistema de Pagos y actualiza estado.	Actualiza Membresia del atleta en RepositorioMembresia a con nuevo estado y fecha de vencimiento.	Secuencia: seleccionar atleta → ingresar pago → procesar transacción externa → actualizar estado → confirmar.
CUC-ADM-02 Mostrar listado de membresías	PantallaMembresias solicita mostrar lista de atletas con estado.	ControlMembresias solicita todos los registros de atletas con su membresía.	RepositorioMembresia devuelve lista de Membresia con estado y fecha de vencimiento por atleta.	Secuencia: solicitar listado → recuperar lista de atletas con membresía → mostrar tabla.
CUC-ENT-01 Registrar rendimiento de WOD	PantallaRendimiento captura atleta, sesión, tiempo, repeticiones, peso y puntuación.	ControlRendimiento valida que el atleta tenga rutinas asignadas, verifica coherencia de datos y coordina el guardado.	Crea Rendimiento y lo guarda en RepositorioRendimiento.	Secuencia: seleccionar atleta → validar rutinas asignadas → ingresar resultados → validar coherencia → crear entidad

				Rendimiento → guardar → confirmar.
CUC-ENT-01 Modificar registro de rendimiento	PantallaRendimiento captura ID de registro y nuevos valores.	ControlRendimiento busca existencia y coordina actualización con validación de coherencia.	Actualiza Rendimiento existente en RepositorioRendimiento.	Secuencia: seleccionar registro → cargar datos actuales → modificar resultados → validar coherencia → actualizar entidad → confirmar.
CUC-ENT-01 Mostrar historial de rendimiento	PantallaRendimiento solicita mostrar historial de un atleta y presenta tabla con evolución.	ControlRendimiento solicita todos los registros del atleta.	RepositorioRendimiento devuelve lista de Rendimiento del atleta seleccionado.	Secuencia: seleccionar atleta → solicitar historial → recuperar lista de registros → mostrar tabla con evolución.
CUC-ATL-01 Reservar clase	PantallaReservas captura clase seleccionada por el atleta desde la vista semanal.	ControlReservas valida membresía vigente, cupos disponibles y límite de reservas, luego coordina el guardado.	Crea Reserva en RepositorioReserva, actualiza cupo en RepositorioClase y valida estado en RepositorioMembresía.	Secuencia: ver horarios disponibles → seleccionar clase → validar membresía → validar cupos → validar límite → crear entidad Reserva →

				guardar → notificar.
CUC-ATL -01 Cancelar reserva	PantallaReservas solicita selección de reserva activa a cancelar.	ControlReservas verifica plazo de cancelación y confirma operación.	Actualiza estado de Reserva a "Cancelada" en RepositorioReserva y libera cupo en RepositorioClase.	Secuencia: ver Mis Reservas → seleccionar reserva → validar plazo → confirmar cancelación → actualizar estado → liberar cupo → notificar.
CUC-ATL -01 Modificar reserva	PantallaReservas captura reserva a cancelar y nueva clase a reservar.	ControlReservas coordina cancelación de reserva actual y creación de nueva con las mismas validaciones.	Actualiza Reserva cancelada y crea nueva Reserva en RepositorioReserva, ajusta cupos en RepositorioClase.	Secuencia: seleccionar reserva actual → cancelar → liberar cupo → seleccionar nueva clase → validar → crear nueva reserva → confirmar.
CUC-ATL -01 Mostrar Mis Reservas	PantallaReservas solicita mostrar lista de reservas activas del atleta.	ControlReservas solicita todos los registros del atleta autenticado.	RepositorioReserva devuelve lista de Reserva activas con horario, entrenador y estado.	Secuencia: solicitar Mis Reservas → recuperar lista de reservas activas → mostrar tabla con horario, entrenador y estado.

Diagramas de secuencia

Administrador

1. CUC-ADM-01: Administrar la Agenda de Clases (Crear nueva clase)

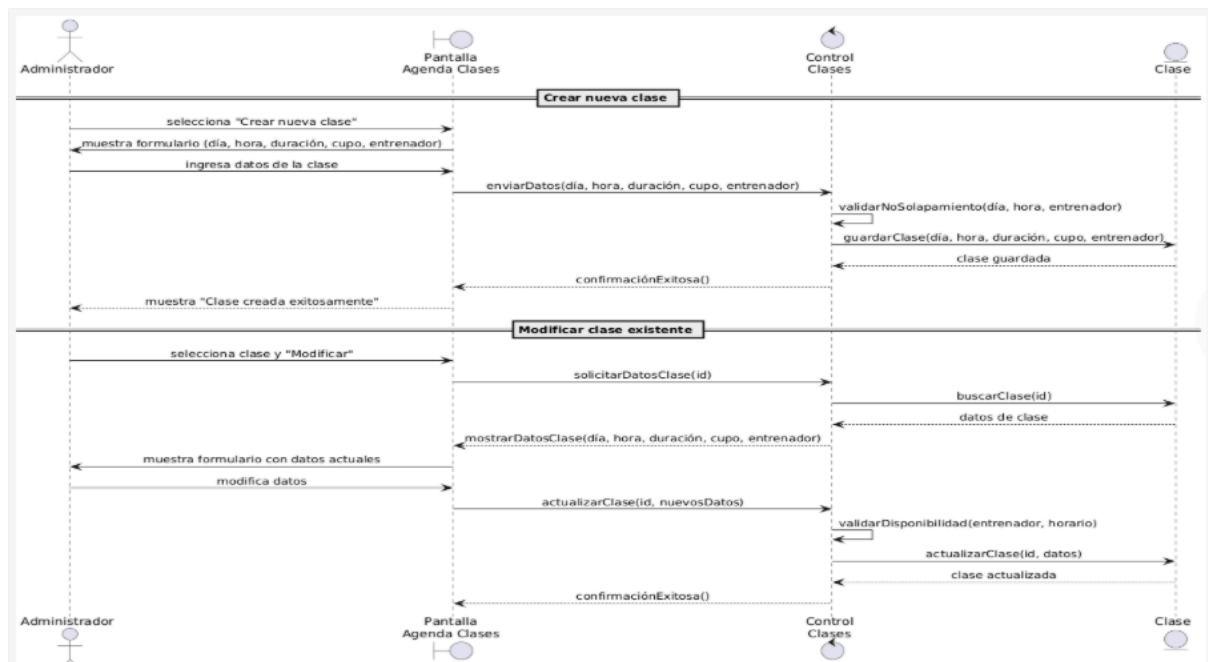
```
@startuml
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

actor "Administrador" as Admin
boundary "Pantalla\nAgenda Clases" as UI
control "Control\nClases" as Ctrl
entity "Clase" as Est

== Crear nueva clase ==
Admin -> UI : selecciona "Crear nueva clase"
UI -> Admin : muestra formulario (día, hora, duración, cupo, entrenador)
Admin -> UI : ingresa datos de la clase
UI -> Ctrl : enviarDatos(día, hora, duración, cupo, entrenador)
Ctrl -> Ctrl : validarNoSolapamiento(día, hora, entrenador)
Ctrl -> Est : guardarClase(día, hora, duración, cupo, entrenador)
Est --> Ctrl : clase guardada
Ctrl --> UI : confirmaciónExitosa()
UI --> Admin : muestra "Clase creada exitosamente"

== Modificar clase existente ==
Admin -> UI : selecciona clase y "Modificar"
UI -> Ctrl : solicitarDatosClase(id)
Ctrl -> Est : buscarClase(id)
Est --> Ctrl : datos de clase
Ctrl --> UI : mostrarDatosClase(día, hora, duración, cupo, entrenador)
UI -> Admin : muestra formulario con datos actuales
Admin -> UI : modifica datos
UI -> Ctrl : actualizarClase(id, nuevosDatos)
Ctrl -> Ctrl : validarDisponibilidad(entrenador, horario)
Ctrl -> Est : actualizarClase(id, datos)
Est --> Ctrl : clase actualizada
Ctrl --> UI : confirmaciónExitosa()

@enduml
```



2. CUC-ADM-02: Gestionar Membresías y Pagos (Crear membresía y registrar pago)

@startuml
 skinparam sequenceMessageAlign center
 skinparam responseMessageBelowArrow true

actor "Administrador" as Admin
 boundary "Pantalla\nMembresías" as UI
 control "Control\nMembresías" as Ctrl
 entity "Membresía" as Est
 actor "Sistema de Pagos" as SP

== Crear tipo de membresía ==

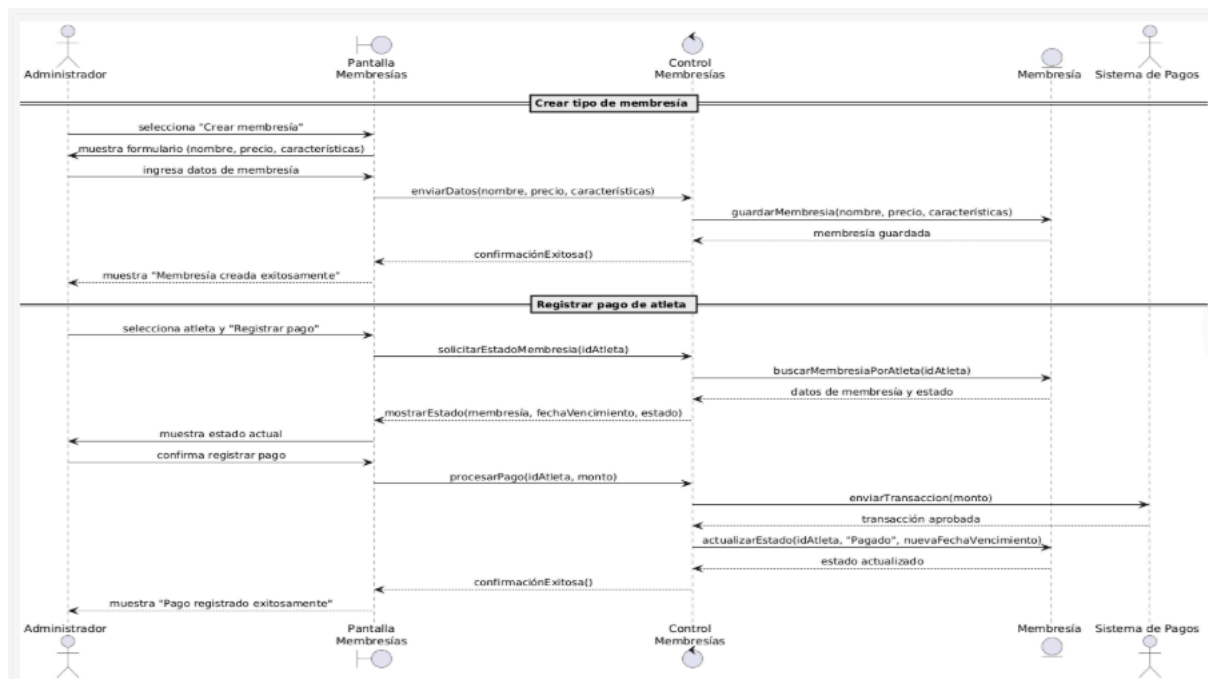
Admin -> UI : selecciona "Crear membresía"
 UI -> Admin : muestra formulario (nombre, precio, características)
 Admin -> UI : ingresa datos de membresía
 UI -> Ctrl : enviarDatos(nombre, precio, características)
 Ctrl -> Est : guardarMembresia(nombre, precio, características)
 Est --> Ctrl : membresía guardada
 Ctrl --> UI : confirmaciónExitosa()
 UI --> Admin : muestra "Membresía creada exitosamente"

== Registrar pago de atleta ==

Admin -> UI : selecciona atleta y "Registrar pago"
 UI -> Ctrl : solicitarEstadoMembresia(idAtleta)
 Ctrl -> Est : buscarMembresiaPorAtleta(idAtleta)
 Est --> Ctrl : datos de membresía y estado
 Ctrl --> UI : mostrarEstado(membresía, fechaVencimiento, estado)

UI -> Admin : muestra estado actual
 Admin -> UI : confirma registrar pago
 UI -> Ctrl : procesarPago(idAtleta, monto)
 Ctrl -> SP : enviarTransaccion(monto)
 SP --> Ctrl : transacción aprobada
 Ctrl -> Est : actualizarEstado(idAtleta, "Pagado", nuevaFechaVencimiento)
 Est --> Ctrl : estado actualizado
 Ctrl --> UI : confirmaciónExitosa()
 UI --> Admin : muestra "Pago registrado exitosamente"

@enduml



Entrenador

3. CUC-ENT-01: Seguimiento del Rendimiento de Atletas

@startuml
 skinparam sequenceMessageAlign center
 skinparam responseMessageBelowArrow true

actor "Entrenador" as Ent
 boundary "Pantalla\nRendimiento" as UI
 control "Control\nRendimiento" as Ctrl
 entity "Rendimiento" as Est
 entity "Atleta" as E_Atl

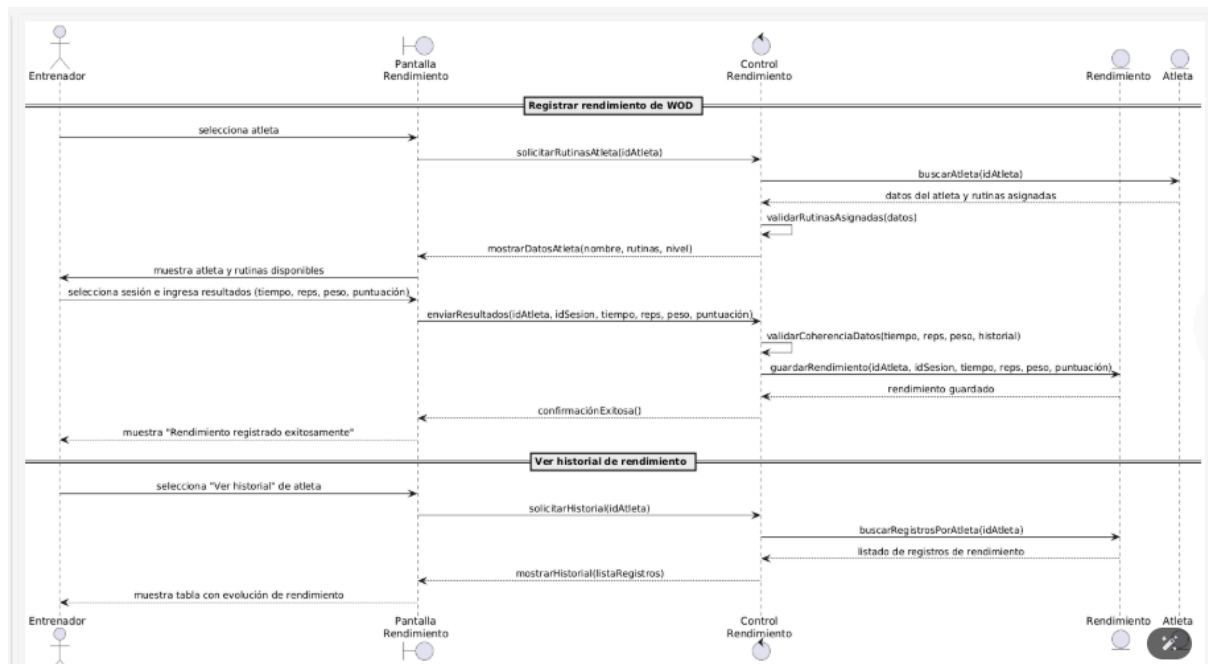
== Registrar rendimiento de WOD ==

Ent -> UI : selecciona atleta
 UI -> Ctrl : solicitarRutinasAtleta(idAtleta)
 Ctrl -> E_Atl : buscarAtleta(idAtleta)
 E_Atl --> Ctrl : datos del atleta y rutinas asignadas
 Ctrl -> Ctrl : validarRutinasAsignadas(datos)
 Ctrl --> UI : mostrarDatosAtleta(nombre, rutinas, nivel)
 UI -> Ent : muestra atleta y rutinas disponibles
 Ent -> UI : selecciona sesión e ingresa resultados (tiempo, reps, peso, puntuación)
 UI -> Ctrl : enviarResultados(idAtleta, idSesion, tiempo, reps, peso, puntuación)
 Ctrl -> Ctrl : validarCoherenciaDatos(tiempo, reps, peso, historial)
 Ctrl -> Est : guardarRendimiento(idAtleta, idSesion, tiempo, reps, peso, puntuación)
 Est --> Ctrl : rendimiento guardado
 Ctrl --> UI : confirmaciónExitosa()
 UI --> Ent : muestra "Rendimiento registrado exitosamente"

== Ver historial de rendimiento ==

Ent -> UI : selecciona "Ver historial" de atleta
 UI -> Ctrl : solicitarHistorial(idAtleta)
 Ctrl -> Est : buscarRegistrosPorAtleta(idAtleta)
 Est --> Ctrl : listado de registros de rendimiento
 Ctrl --> UI : mostrarHistorial(listaRegistros)
 UI --> Ent : muestra tabla con evolución de rendimiento

@enduml



Atleta

4. CUC-ATL-01: Gestionar Reservas de Clases

```

@startuml
skinparam sequenceMessageAlign center
skinparam responseMessageBelowArrow true

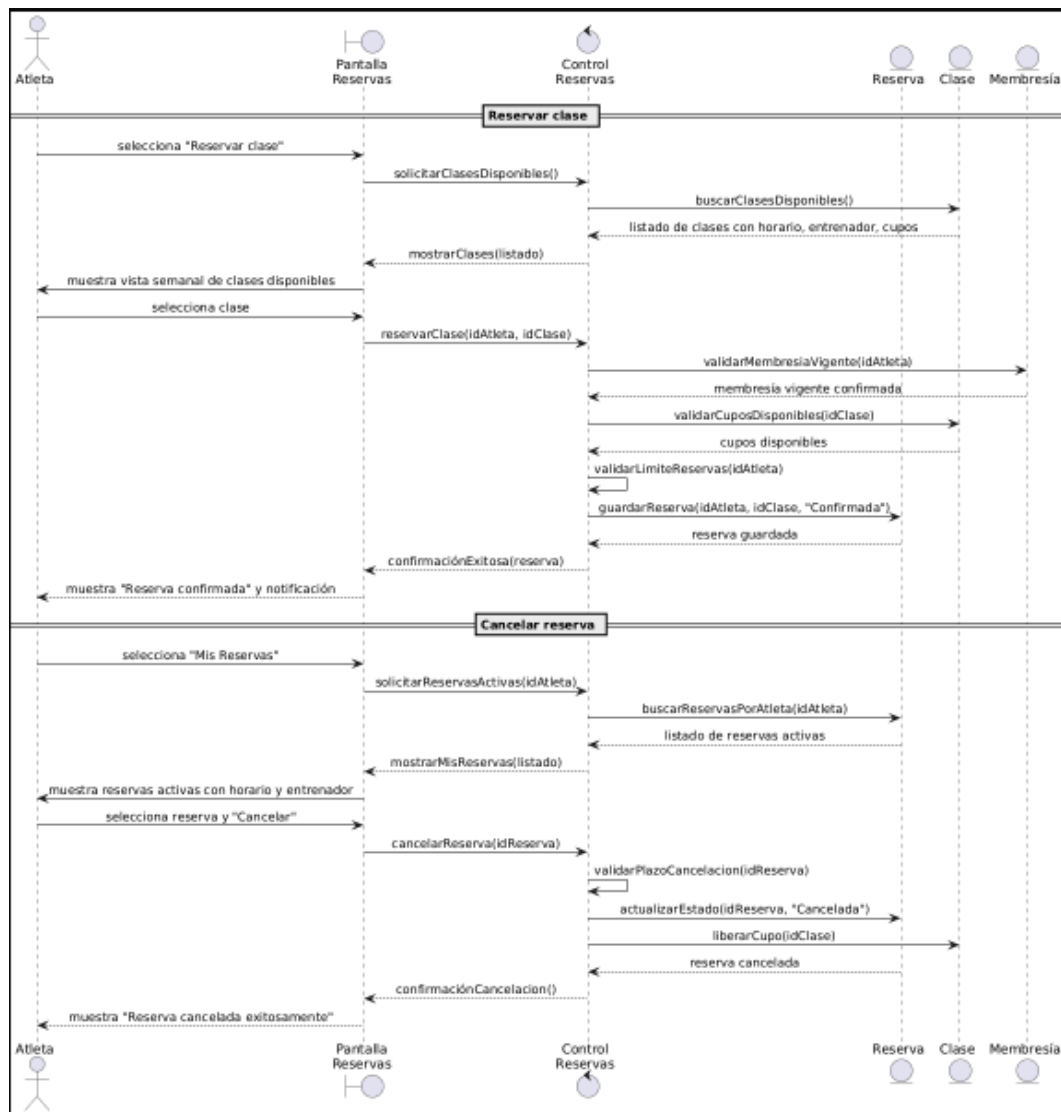
actor "Atleta" as Atl
boundary "Pantalla\nReservas" as UI
control "Control\nReservas" as Ctrl
entity "Reserva" as E_Res
entity "Clase" as E_Cls
entity "Membresía" as E_Memb

== Reservar clase ==
Atl -> UI : selecciona "Reservar clase"
UI -> Ctrl : solicitarClasesDisponibles()
Ctrl -> E_Cls : buscarClasesDisponibles()
E_Cls --> Ctrl : listado de clases con horario, entrenador, cupos
Ctrl --> UI : mostrarClases(listado)
UI -> Atl : muestra vista semanal de clases disponibles
Atl -> UI : selecciona clase
UI -> Ctrl : reservarClase(idAtleta, idClase)
Ctrl -> E_Memb : validarMembresiaVigente(idAtleta)
E_Memb --> Ctrl : membresía vigente confirmada
Ctrl -> E_Cls : validarCuposDisponibles(idClase)
E_Cls --> Ctrl : cupos disponibles
Ctrl -> Ctrl : validarLimiteReservas(idAtleta)
Ctrl -> E_Res : guardarReserva(idAtleta, idClase, "Confirmada")
E_Res --> Ctrl : reserva guardada
Ctrl --> UI : confirmaciónExitosa(reserva)
UI --> Atl : muestra "Reserva confirmada" y notificación

== Cancelar reserva ==
Atl -> UI : selecciona "Mis Reservas"
UI -> Ctrl : solicitarReservasActivas(idAtleta)
Ctrl -> E_Res : buscarReservasPorAtleta(idAtleta)
E_Res --> Ctrl : listado de reservas activas
Ctrl --> UI : mostrarMisReservas(listado)
UI -> Atl : muestra reservas activas con horario y entrenador
Atl -> UI : selecciona reserva y "Cancelar"
UI -> Ctrl : cancelarReserva(idReserva)
Ctrl -> Ctrl : validarPlazoCancelacion(idReserva)
Ctrl -> E_Res : actualizarEstado(idReserva, "Cancelada")
Ctrl -> E_Cls : liberarCupo(idClase)
E_Res --> Ctrl : reserva cancelada
Ctrl --> UI : confirmaciónCancelacion()
UI --> Atl : muestra "Reserva cancelada exitosamente"

@enduml

```



Conclusión técnica

El diagrama de clases de análisis consolida la arquitectura preliminar del sistema IRONCLAD BOX, integrando los 4 casos base priorizados (CUC-ADM-01, CUC-ADM-02, CUC-ENT-01 y CUC-ATL-01) mediante el patrón de separación de responsabilidades en tres capas: Frontera, Control y Entidad. Se identificaron 4 clases de frontera (PantallaAgendaClases, PantallaMembresías, PantallaRendimiento, PantallaReservas) que encapsulan la interacción con los actores Administrador, Entrenador y Atleta; 4 clases de control (ControlClases, ControlMembresías, ControlRendimiento, ControlReservas) que coordinan la lógica de negocio y las validaciones automáticas del sistema; y 4 entidades de dominio (Clase, Membresía, Rendimiento, Reserva) que representan la información persistente en la base de datos MySQL. Adicionalmente, se modeló el actor externo Sistema de Pagos, evidenciando la dependencia con un tercero para el procesamiento de transacciones.

El análisis revela dos relaciones cruzadas relevantes para el diseño detallado:

ControlReservas depende tanto de Membresía (para validar vigencia) como de Clase (para validar cupos disponibles), lo que implica que la entidad Reserva mantiene asociaciones con ambas entidades y requerirá integridad referencial en el modelo relacional. La entidad Clase es la única compartida entre dos módulos distintos (Administrador y Atleta), por lo que debe diseñarse con cuidado para soportar operaciones concurrentes, especialmente en escenarios de alta demanda como la apertura de reservas. Las clases de control concentran las validaciones definidas en el SRS como responsabilidad del actor "Sistema", entre ellas: no solapamiento de horarios, membresía vigente, cupos disponibles, coherencia de datos de rendimiento y límites de reservas simultáneas.